

EasyMemories

Aplicación móvil de recuerdos para eventos. El anfitrión crea un "espacio" (fiesta, boda, XV años, etc.) y genera un código QR; los invitados lo escanean para entrar sin necesidad de cuenta ni contraseña, y suben fotos con un comentario que quedan en un muro de recuerdos compartido del evento.

Tecnologías

- **Backend:** .NET 10 (ASP.NET Core Web API, patrón MVC), Entity Framework Core + SQLite, ASP.NET Core Identity, JWT
- **Frontend móvil:** React Native con Expo (TypeScript), React Navigation
- **Almacenamiento de fotos:** Azure Blob Storage (URLs de lectura firmadas con SAS, sin acceso anónimo)
- **Generación de QR:** QRCode

Arquitectura



- El **admin** se autentica con correo/contraseña (JWT) y administra sus espacios.
- El **invitado** no tiene cuenta: el código de unión (obtenido al escanear el QR) es su único método de acceso al espacio.
- Las fotos nunca son públicas por URL directa: cada lectura se sirve con una URL SAS de Azure con expiración, respetando que la cuenta de Storage tiene deshabilitado el acceso anónimo a blobs.

Estructura del repositorio

```

EasyMemories/
├── Controllers/      Endpoints de la API (Auth, Spaces, Join)
├── Models/           Entidades (AppUser, Space, Memory)
├── Data/             DbContext (EF Core)
├── Services/         Blob Storage, JWT, generación de QR
├── DTOs/             Contratos de request/response
├── Migrations/       Migraciones de EF Core
├── mobile/           App Expo / React Native
│   └── src/
│       ├── api/      Cliente HTTP (auth, spaces, join)
│       ├── context/  Sesión del admin (AuthContext)
│       ├── navigation/ Rutas y deep linking
│       └── screens/   Pantallas de admin e invitado

```

Cómo correr el proyecto

Backend

```

cd EasyMemories
dotnet user-secrets set "AzureBlobStorage:ConnectionString" "<connection string de tu
dotnet run

```

Debe iniciar en <http://0.0.0.0:5215> (Swagger disponible en </swagger>).

App móvil

```

cd EasyMemories/mobile
npx expo start

```

Escanea el QR con la app **Expo Go** desde tu celular, conectado a la misma red WiFi que la PC.

Modelo de datos

Entidad	Descripción
AppUser	Cuenta del anfitrión/admin (ASP.NET Identity)
Space	Un evento: nombre, descripción, código de unión único, dueño
Memory	Una foto + comentario opcional subido por un invitado a un Space

Endpoints principales

Método	Ruta	Acceso	Descripción
POST	<code>/api/auth/register</code>	Público	Crear cuenta de anfitrión
POST	<code>/api/auth/login</code>	Público	Iniciar sesión → JWT
POST	<code>/api/spaces</code>	Admin	Crear espacio (genera código QR)
GET	<code>/api/spaces</code>	Admin	Listar espacios propios
GET	<code>/api/spaces/{id}/qrcode</code>	Admin	PNG del QR del espacio
GET	<code>/api/join/{joinCode}</code>	Público	Info del espacio para el invitado
POST	<code>/api/join/{joinCode}/memories</code>	Público	Subir foto + comentario
GET	<code>/api/join/{joinCode}/memories</code>	Público	Muro de recuerdos del espacio

Equipo y áreas de trabajo

Integrante	Área
Danna	Arquitectura del backend: modelado de datos (<code>AppUser</code> , <code>Space</code> , <code>Memory</code>), configuración de EF Core + SQLite y estructura MVC del proyecto en <code>Program.cs</code> .
Naomi	Autenticación y seguridad: <code>AuthController</code> , integración de ASP.NET Identity, emisión y validación de JWT (<code>TokenService</code>).
Diego	Integración con Azure: <code>BlobStorageService</code> (subida de fotos y URLs firmadas SAS) y <code>QrCodeService</code> (generación del código QR de cada espacio).
Charli	App móvil — pantallas y navegación: flujo de anfitrión (login, crear espacio, ver QR) e invitado (unirse, muro de recuerdos), <code>RootNavigator</code> y tema visual.
Diego	App móvil — integración con la API: cliente HTTP (<code>api/</code>), escaneo de QR con <code>expo-camera</code> , captura/selección de foto con <code>expo-image-picker</code> y subida multipart.
Jorge	Pruebas de extremo a extremo (registro, creación de espacio, subida de fotos contra Azure real), documentación y configuración de red/firewall para pruebas en dispositivo físico.

Notas de seguridad

- La cuenta de Storage mantiene **"Blob anonymous access" deshabilitado**; todas las fotos se sirven con URLs SAS de lectura con expiración, nunca públicas de forma permanente.
- Las contraseñas de los administradores se guardan con hash vía ASP.NET Core Identity, nunca en texto plano.

- La cadena de conexión de Azure y la clave de firma JWT se manejan con `dotnet user-secrets`, nunca se commitean al repositorio.